



汇编语言

5 传送类指令

张青

ieqzhang@zzu.edu.cn

教学内容



- 寻址方式

教材章节：3.5

慕课：4.5

- ▶ 识别寻址方式

- 传送类指令

教材章节：4.2,4.3

慕课：5.3

- ▶ MOV指令

- ▶ XCHG、LEA指令

- ▶ PUSH和POP指令

学习目标



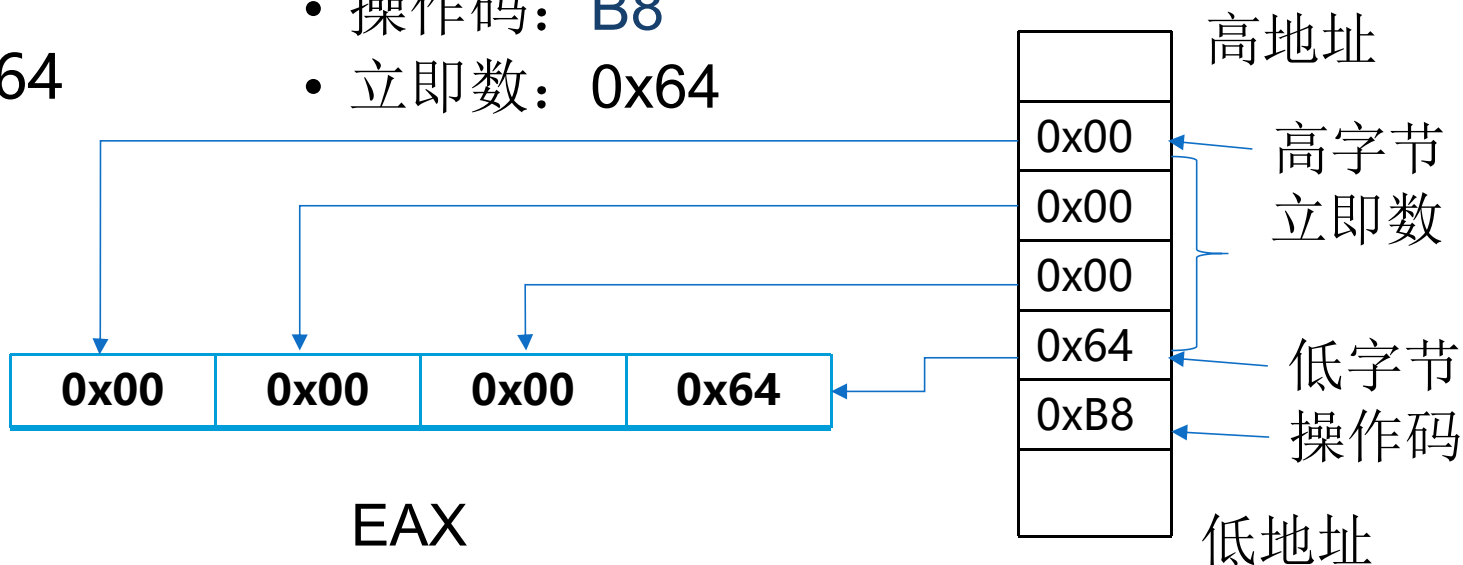
- 能正确书写数据传送指令
- 能编写指令序列完成数据传送
- 能写出C语言程序对应的汇编指令
- 能写出汇编指令序列对应的C语句
- 能用GDB调试程序
- 养成细致严谨的态度，注意C语言与汇编语言之间的关系，培养系统观

数据寻址方式

- 指令有两部分：操作码和操作数
 - 操作码：处理器要执行哪种操作
不可缺少，用助记符表示
 - 操作数：指令执行的参与者
各种操作的对象，需要通过地址指示
- 数据寻址方式：通过地址查找数据（操作数）
 - 立即数寻址：数据在指令代码中，用常量表达
 - 寄存器寻址：数据在寄存器中，用寄存器名表示
 - 存储器寻址：数据在主存中，用存储器地址代表

立即数寻址

- 操作数紧跟操作码，是机器代码的一部分
- 操作数从指令代码中立即得到，即立即数 (Immediate) ，用常量形式直接表达
- 立即数寻址方式只用于源操作数，常用来给寄存器和存储单元赋值
- 例如：**mov eax, 100**
 - 机器代码：B8 64 00 00 00
 - 操作码：B8
 - 立即数：0x00000064
- **mov al, 100**
 - 机器代码：B8 64
 - 操作码：B8
 - 立即数：0x64



寄存器寻址

- 操作数存放在处理器的内部寄存器中
- 用寄存器名表示它的内容
- 绝大多数指令采用通用寄存器寻址
- 部分指令支持专用寄存器，例如段寄存器
- 寄存器寻址方式简单快捷，最常使用
- 例如：MOV EBX,EAX

64位通用寄存器：RAX RBX RCX RDX

32位通用寄存器：EAX EBX ECX EDX

16位通用寄存器：AX BX CX DX

8位通用寄存器：AH AL BH BL

- 操作数在主存中，通过存储器地址指示
- 编程时，存储器地址使用包含段选择器和偏移地址的逻辑地址
 - 段选择器（段寄存器）指示段基地址
 - 默认规定：数据在DS指向的数据段；EBP或ESP作为基地址，数据在SS指向的堆栈段
 - 显式说明：使用段超越指令前缀，段寄存器名后跟英文冒号
 - 偏移地址由各种寻址方式计算
 - 常被称为**有效地址EA**（Effective Address）

偏移地址的组成

64位有效地址 = 基址寄存器 + (变址寄存器 × 比例) + 位移量

- 基址寄存器：任何一个64位通用寄存器之一，RIP
- 变址寄存器：除RSP外的任何64位通用寄存器之一
- 比例：1, 2, 4, 8或16
- 位移量：8, 16, 32或64位有符号值



变化出多种主存寻址方式

1. 直接寻址

- 有效地址只有位移量部分，直接包含在指令代码中
- 常用于存取变量
- 仅适用于IA-32处理器保护模式
- 64位模式下不使用这种寻址方式
- 例如：在IA-32处理保护模式下

MOV ECX,[COUNT] #COUNT是变量

MOV ECX, DS:[405000H]

- 指令代码：8B 0D 00 50 40 00
- 操作码和寻址方式：8B 0D
- 操作数：有效地址 00405000H

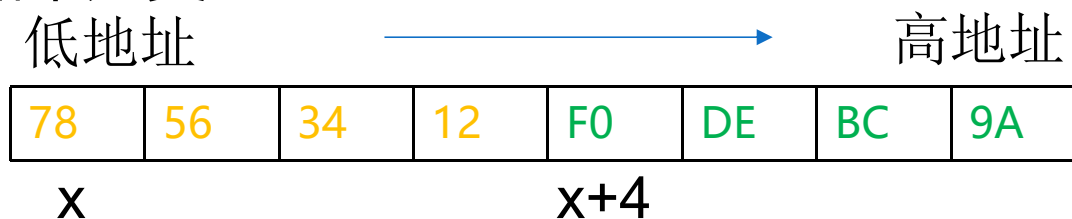
2.RIP相对寻址方式

- 64位模式下最常使用的一种存储器寻址方式
- 有效地址 = RIP+64位位移量

行数	线性地址	机器码	汇编语句
			.data
9	0000	78563412	x: .long 0x12345678,0x9ABCDEF0
9		F0DEBC9A	
10	0008	31323334	y: .ascii "12345"
10		35	
			.text
14	0000	8B050000	mov eax,x[rip] /*有效地址=x+rip,eax=0x12345678 */
14		0000	
17	0010	8B050800	mov eax,y[rip] /*有效地址=y+rip,eax=0x34333231 */
17		0000	
20	0020	488B0504	mov rax,x[rip+4] /*有效地址=x+rip+4,rax=0x343332319ABCDEF0 */
20		000000	

3. 寄存器间接寻址

- 有效地址存放在寄存器中(寄存器内容 = 偏移地址 = 有效地址)
- 用中括号括起寄存器
- 可以方便地对数组的元素或字符串的字符进行操作
- 寄存器间接寻址没有说明存储单元类型
- 例如:



```
.data
x: .long 0x12345678,0x9ABCDEF0
.text
lea rax, x[rip] /*x地址复制到rax*/
mov bl, [rax] /*有效地址=rax, bl=0x78 */
mov bx, [rax] /*有效地址=rax, bx=0x5678 */
mov ebx,[rax] /*有效地址=rax, ebx=0x12345678 */
mov rbx,[rax] /*有效地址=rax, rax=0x9ABCDEF012345678 */
```

4. 寄存器相对寻址

- 有效地址是寄存器内容与位移量之和
- 可以方便地对数组的元素或字符串的字符进行操作
- 例如:

```
.data  
x: .long 0x12345678, 0x9ABCDEF0
```

```
.text
```

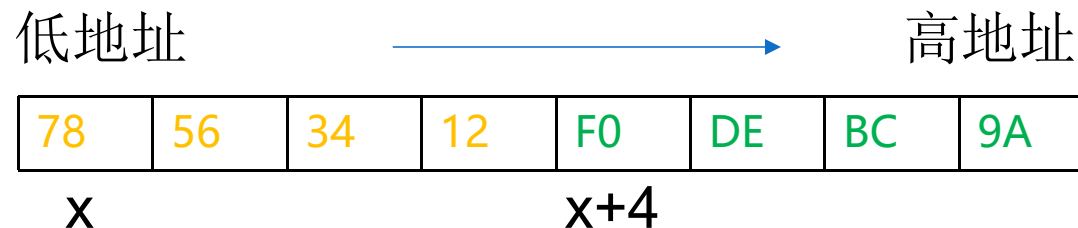
```
lea rax, x[rip] /*x地址复制到rax*/
```

```
mov bl, [rax+1] /*有效地址=rax+1, bl=0x56 */
```

```
mov bx, [rax+2] /*有效地址=rax+2, bx=0x1234 */
```

```
mov ebx, [rax+4] /*有效地址=rax+3, ebx =0x9ABCDEF0 */
```

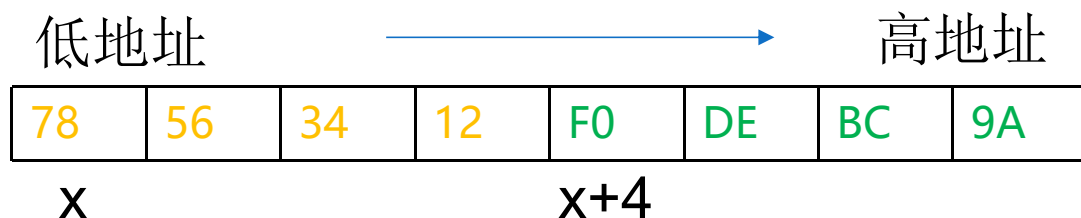
```
mov rbx, [rax+4] /*有效地址=rax+4, rbx =0x000000009ABCDEF0*/
```



主存以字节为可寻址单位
地址的加减是以字节为单位

5. 基址变址寻址

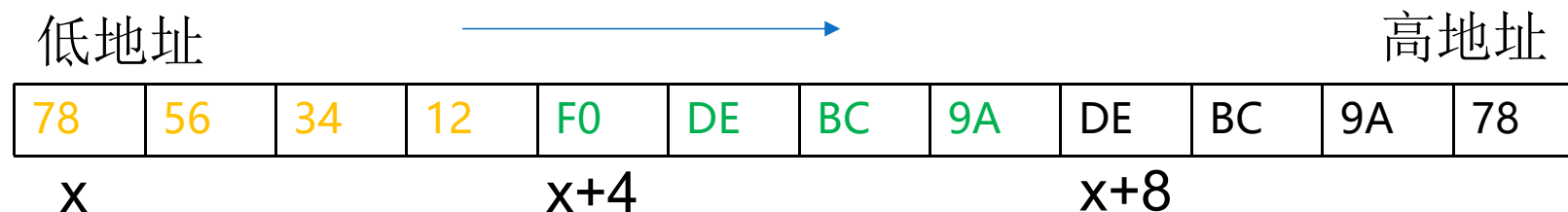
- 使用变址寄存器寻址操作数
- 便于支持二维数组等数据结构
- 例如：



```
.data
x: .long 0x12345678, 0x9ABCDEF0
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 0
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x12345678 */
mov rbx, 4
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x9ABCDEF0 */
```

6.相对基址变址寻址

- 有效地址由基址寄存器的内容加上变址寄存器内容再加上一个64位的位移量构成
 - 有效地址 = 基址寄存器 + 变址寄存器 + 64位位移量**



.data

x: .long 0x12345678, 0x9ABCDEF0, 0x789ABCDE

.text

lea rax, x[rip] /*x地址复制到rax*/

mov rbx, 4

mov eax, [rax+rbx+2] /*有效地址=rax+rbx+2, eax = 0xBCDE9ABC */

mov eax, [rax+rbx+4] /*有效地址=rax+rbx+4, eax = 0x789ABCDE */

7. 带比例的基址变址寻址

- 变址寄存器内容乘以比例1（可省略），2，4或8的变址寻址
- 比例1、2、4和8对应8、16、32和64位数据的字节个数，便于以数组元素为单位寻址相应数据

- 例如：

78	56	34	12	F0	DE	BC	9A	9A	78	56	34
----	----	----	----	----	----	----	----	----	----	----	----

低地址 → 高地址

x x+4 x+8

```
.data
x: .long 0x12345678, 0x9ABCDEF0, 0x3456789A
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 1
mov eax,[rax+rbx*4] /*有效地址=rax+rbx+2, eax = 0x9ABCDEF0 */
mov eax,[ rax+rbx*4+4] /*有效地址=rax+rbx+4, eax = 0x3456789A */
```

例3-9 数组访问

```
//eg0309.c
#include <stdio.h>
int array[]={12345,-12345};
int i,*pi=array;
int main(){
    printf("%d\n",array[0]);
    i=1;
    printf("%d\n",array[i]);
    i=2;
    printf("%d\n",array[i]);
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    return 0;
}
```

mov	edx, DWORD PTR array[rip]	#RIP相对寻址, array[0]
mov	edx, DWORD PTR array[rip+4]	#RIP相对寻址, array[1]
mov	edx, DWORD PTR array[rip+8]	#RIP相对寻址, array[2]
mov	rax, QWORD PTR pi[rip]	/*rax=pi*/
mov	edx, DWORD PTR [rax]	/*寄存器间接寻址, *pi */
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rdx=pi+4
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+1)
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rax=pi+8
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+2)

讨论：源操作数的寻址方式是什么？



//addressing.c

```
.....  
int main(){  
    int i;  
    char *p;  
    f=1.0;  
    for(i=0;i<5;i++)  
        array[i]+=10;  
    for(p=str;*p;p++)  
        *p+=1;  
    return 0;  
}
```

mov **DWORD PTR f[rip], 0x3f800000**

.....
mov **eax, 0**

.....
lea **rcx, array[rip]**
movsx **r8, eax**
mov **edx, DWORD PTR [rcx+r8*4]**
add **edx, 10**

.....
lea **rdx, str[rip]**
add **eax, 1**
mov **BYTE PTR [rdx], al**
add **rdx, 1**
movzx **eax, BYTE PTR [rdx]**

....

立即数寻址

立即数寻址

RIP相对寻址

寄存器寻址

带比例基址变址寻址

立即数寻址

RIP相对寻址

立即数寻址

寄存器寻址

立即数寻址

寄存器间接寻址

gcc -S -Og -masm=intel -c -o addressing.S addressing.c

寻址的数据是什么?



dvar: .long 0x12345678,12

```
lea rbx, dvar[rip]
mov eax, [rbx]
mov ecx, dvar[rip]
mov rsi, 2
mov edi, [rbx+rsi*2]
mov edx, [rbx+rsi+2]
mov al, byte ptr dvar[rip+2]
```

RBX = 0x00405000

EAX = 0x12345678

ECX = 0x12345678

RSI = 2

EDI = 0x0000000C

EDX = 0x0000000C

AL = 0x34

dvar → 0x00405000

→ 0x00405004

+4

地址

00

00

00

0C

12

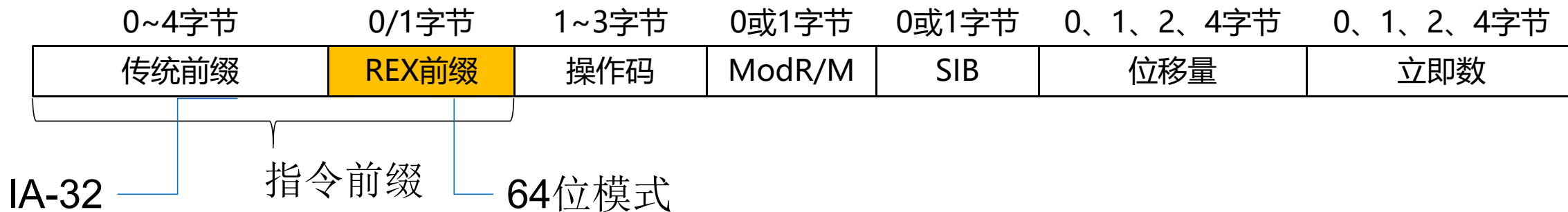
34

56

78

X86处理器指令格式 (Instruction Format)

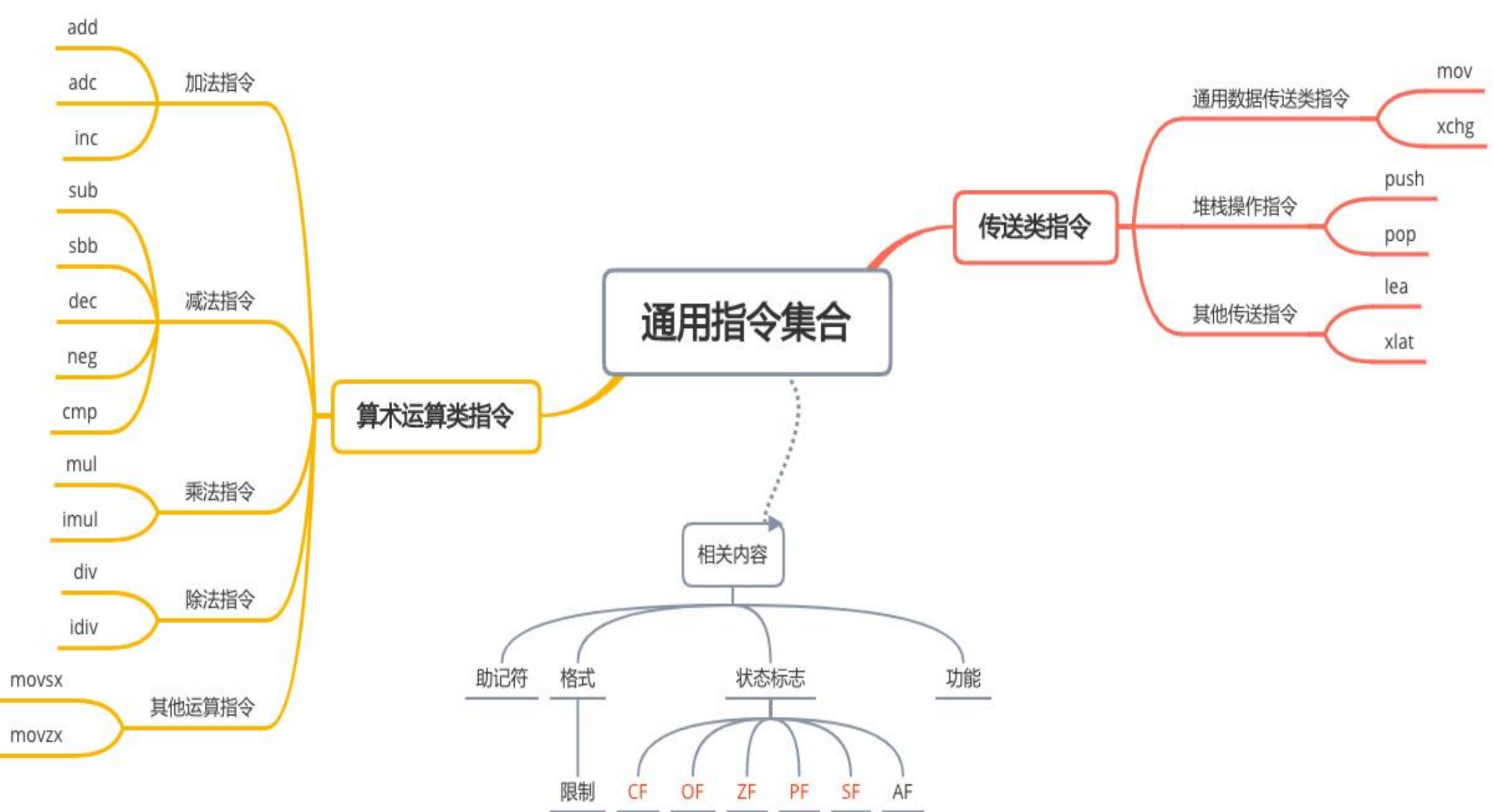
- x86处理器采用可变长度指令格式 (复杂指令集合)
- 操作码
 - 可选的指令前缀 (用于扩展指令功能)
 - 1 ~ 3字节的主要操作码
- 操作数
 - 可选的寻址方式域 (包括ModR/M和SIB字段)
 - 可选的位移量
 - 可选的立即数





学习指令的注意事项

- **指令的功能**——该指令能够实现何种操作。通常指令助记符就是指令功能的英文单词或其缩写形式
- **指令支持的寻址方式**——该指令中的操作数可以采用何种寻址方式
- **指令对标志的影响**——该指令执行后是否对各个标志位有影响，以及如何影响
- **其他方面**——该指令其他需要特别注意的地方，如指令执行时的约定设置、必须预置的参数、隐含使用的寄存器等



数据传送类指令

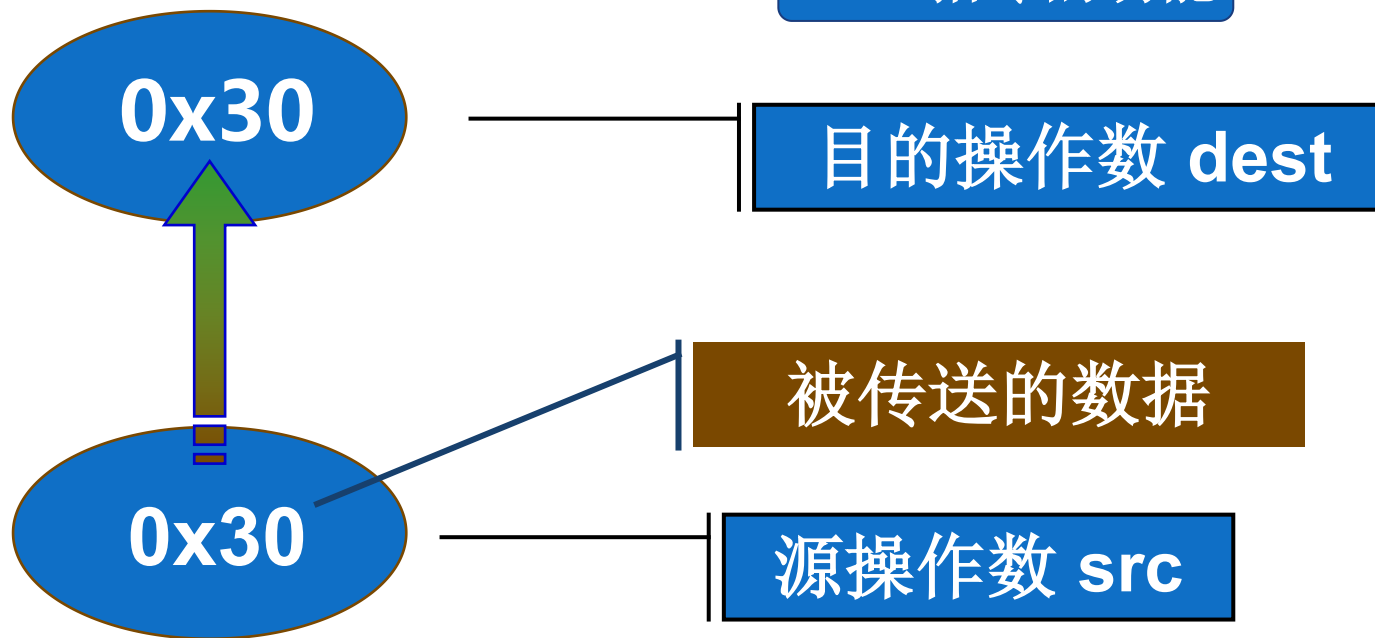
- 数据传送
 - 把数据从一个位置传送到另一个位置
 - 计算机中最基本的操作
 - 程序设计中最常使用的指令
- 除标志寄存器传送指令外，均不影响标志位

全面而准确地理解每条指令的功能和应用是编写汇编语言程序的关键也是理解处理器如何进行数据处理的核心

通用数据传送指令

- 提供方便灵活的通用数据传送操作
- 主要有传送MOV和交换XCHG指令
- 地址传送指令LEA
- 堆栈操作指令PUSH, POP

MOV指令的功能



传送指令MOV



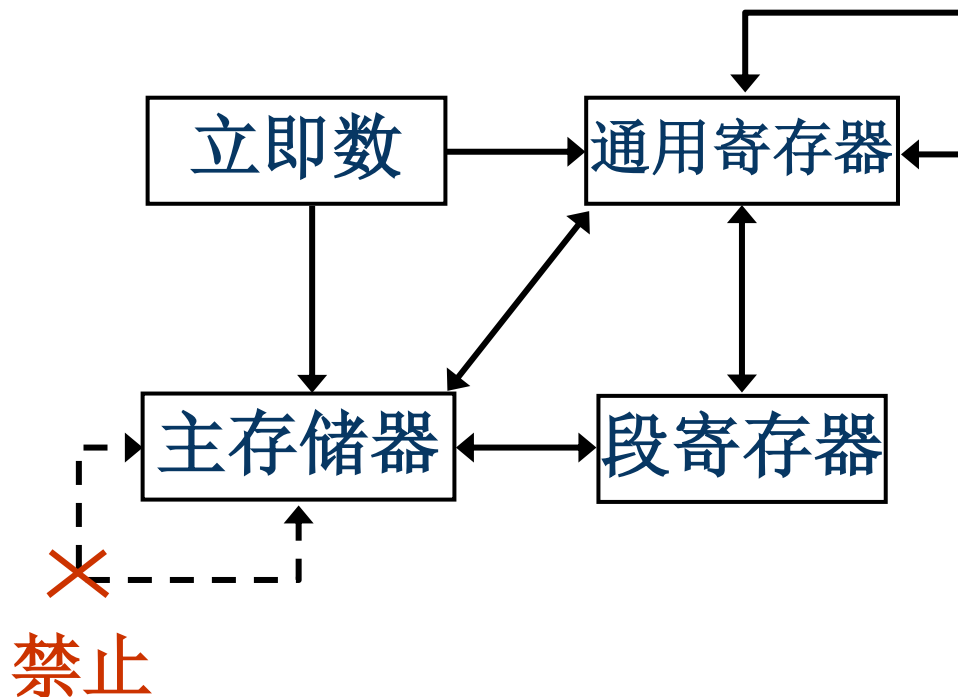
- 把一个字节、字、双字、四字的操作数从源位置传送至目的位置

MOV reg/mem,imm

MOV reg/mem/seg,reg

MOV reg/seg,mem

MOV r16/m16,seg



〔例4-1〕：C语言中的赋值语句



```
/* eg0401.c*/  
long exchange(long *xp, long y)  
{  
    long x = *xp;  
    *xp = y;  
    return x;  
}
```



xp: rcx, x: eax, y: edx

```
mov    eax, DWORD PTR [rcx]  
mov    DWORD PTR [rcx], edx
```

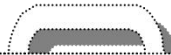
交换指令XCHG

- 将源操作数和目的操作数内容交换

XCHG reg,reg/mem

XCHG reg/mem,reg

- 通用寄存器与通用寄存器之间
- 通用寄存器或存储器之间
- 空操作指令NOP (= XCHG EAX,EAX)
 - 处理器执行空操作该指令，需要花费时间，在主存中占用一个字节空间
 - 机器码：90
 - 实现短时间延时
 - 临时占用代码空间



```
xchg esi,edi  
xchg esi,[rdi]  
xchg al, var[rip]  
xchg rax,var[rip]
```

交换指令示例-大小端转换



.data

x: .long 0x23430a7b

.text

```
mov al, byte ptr x[rip]
xchg al, byte ptr x[rip+3]
xchg byte ptr x[rip], al
mov al, byte ptr x[rip+1]
xchg al, byte ptr x[rip+2]
xchg al, byte ptr x[rip+1]
```

23	43	0a	7b
x	x+1	x+2	x+3

#取第1个字节
#与第4个字节交换
#实现低1、4个字节互换
#同上，AL=第2个字节
#与第3个字节交换，AL=第3个字节
#实现第2、3个字节互换

- 拼写错误、多余的空格、不正确的标点、太过复杂的常量或表达式
- 操作数类型不匹配、无明确类型、错用（段）寄存器、两个存储器操作数、超出数据范围.....

注意1：双操作数必须类型一致

<code>mov esi,dl</code>	#错误：类型不一致 #esi为32位寄存器，dl为8位寄存器
<code>mov esi,edx</code>	#正确：两个32位寄存器传送
<code>mov al,0x050a</code>	#错误：类型不一致 #0x050a超过了寄存器AL范围
<code>mov eax,#050a</code>	#正确：双字量数据传送



注意2：操作数必须有明确的类型

mov [rbx], 255 #错误：无明确类型

mov byte ptr [rbx],255 #正确：BYTE PTR说明是字节操作

mov word ptr [ebx],255 #正确：WORD PTR说明是字操作

mov dword ptr [ebx],255 #正确：DWORD PTR说明是双字操作

注意3：双操作数不允许都是主存单元



#假设dbuf1和dbuf2是两个双字变量

`mov dbuf2[rip],dbuf1[rip]` #错误：两个操作数都是存储单元

`mov eax,dbuf1[rip]` #正确：`eax = dbuf1`（将dbuf1内容送eax）

`mov dbuf2[rip],eax` #正确：`dbuf2 = eax`（将eax内容送dbuf2）

注意4：不可随意操作专用寄存器

mov ds,100

#错误：立即数不能直接传送段寄存器

mov ax,cs

mov ds,ax

#正确：通过AX间接传送给DS

注意5：新字节寄存器的用法



- 不能同时使用传统的高字节和新的字节寄存器

<code>mov dil, al</code>	#正确，新字节寄存器和传统低字节寄存器
<code>mov dil, ah</code>	#错误：不能同时使用 <code>ah</code> 和新增字节寄存器
<code>mov bl, ah</code>	#正确，两个传统字节寄存器
<code>mov dil, r8b</code>	#正确，两个新字节寄存器

课堂练习：下面指令是否正确

- 1 mov dvar[rip+4],dvar[rip]
- 2 mov edi,si
- 3 mov eax,0x50a
- 4 mov al,0x50a
- 5 mov dword ptr [rbx],255
- 6 mov [rbx+4],200
- 7 mov ds, 100
- 8 mov ax, ds
- 9 mov edx, [ebx+0xff]
- 10 mov [rsi], dword ptr [rdi]
- 11 mov ecx, cs
- 12 xchg al, byte ptr dvar[rip+3]
- 13 xchg 100, al

- | | |
|----|---|
| 1 | × |
| 2 | × |
| 3 | √ |
| 4 | × |
| 5 | √ |
| 6 | × |
| 7 | × |
| 8 | √ |
| 9 | √ |
| 10 | × |
| 11 | × |
| 12 | √ |
| 13 | × |



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

课堂练习

邀请码: 97974770

APP首页右上角输入



课堂练习



该邀请码2026年08月26日前有效

计算机类2024级9班10班

地址传送指令

- 地址传送指令获取存储器操作数的地址
LEA r16/r32/r64,mem
#r16/r32/r64←mem的有效地址EA (不需类型一致)
#r16:实地址模式, r32: IA-32e , r64: 64位模式
- LEA指令类似的地址操作符OFFSET的作用
 - LEA指令在指令执行时计算出偏移地址
 - OFFSET操作符在汇编阶段取得变量的偏移地址
 - OFFSET无需在执行时计算、指令执行速度更快
 - LEA指令能获取汇编阶段无法确定的偏移地址

lea rsi, var[rip] #64位模式推荐

mov rdi, offset var #仅用于intel语法, 不稳定, 避免使用

地址传送程序



#数据段

dvar: .long 0x41424344

#代码段

mov eax,dvar[rip]

#EAX = 0x41424344

lea rsi,dvar[rip]

#RSI指向DVAR

mov ebx,[rsi]

#EBX = 0x41424344

lea rdi,dvar[rip]

#RDI指向DVAR

mov ecx,[rdi]

#ECX = 0x41424344

lea edx,[esi+edi*4+0x100] #EDX = ESI + EDI×4 + 0x100

〔例4-2〕：LEA指令实现算术运算

地址传送指令经常用于表达式求值。

```
/*eg0402.c*/  
int scale(int x, int y, int z){  
    long t=x+4*y+12*z;  
    return t;  
}
```

x: rcx
y: rdx
z: r8



```
lea    eax, [rcx+rdx*4]    #eax= x+4*y  
lea    edx, [r8+r8*2]     #edx=3*z  
lea    eax, [rax+rdx*4]    #eax=x+4*y+(3*z)*4
```

进栈指令PUSH



PUSH r16/m16/i16/seg

① $RSP = RSP - 2$ ② $SS:[RSP] = r16/m16/i16/seg$

PUSH r64/m64/i8/i32/i64

① $RSP = RSP - 8$ ② $SS:[RSP] = r64/m64/i8/i32/i64$

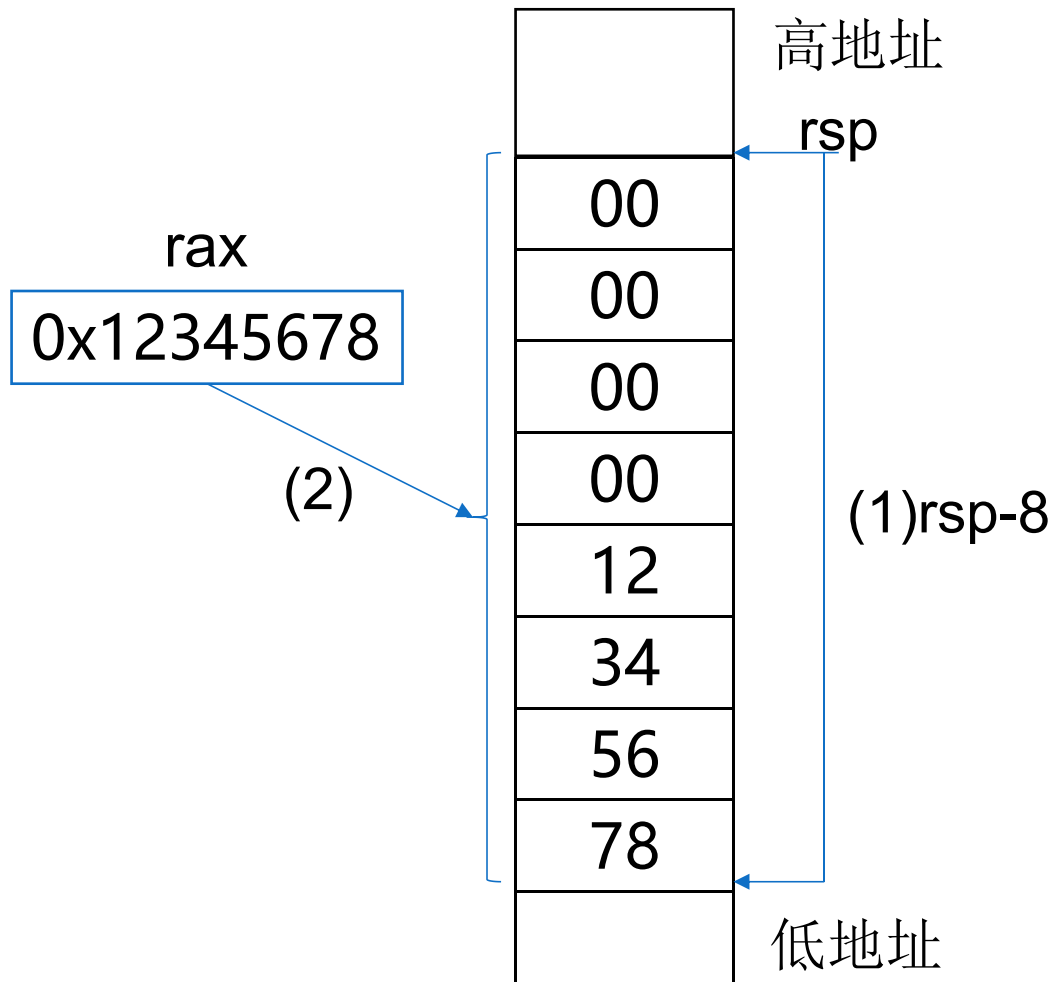
- 先将RSP减小作为当前栈顶
- 后将源操作数(立即数、通用寄存器和段寄存器内容或存储器操作数)传送到当前栈顶
- 以字或四字为单位操作
 - 进栈四字量数据时, RSP减8
 - 进栈字量数据时, RSP减2

push rax

进栈操作



#进栈指令
push rax
#等同于如下两条指令
sub rsp, 8
mov [rsp], rax



出栈指令POP



POP r16/m16/seg

① r16/m16/seg = SS:[RSP] ② RSP = RSP + 2

POP r64/m64

① r64/m64 = SS:[RSP] ② RSP = RSP + 8

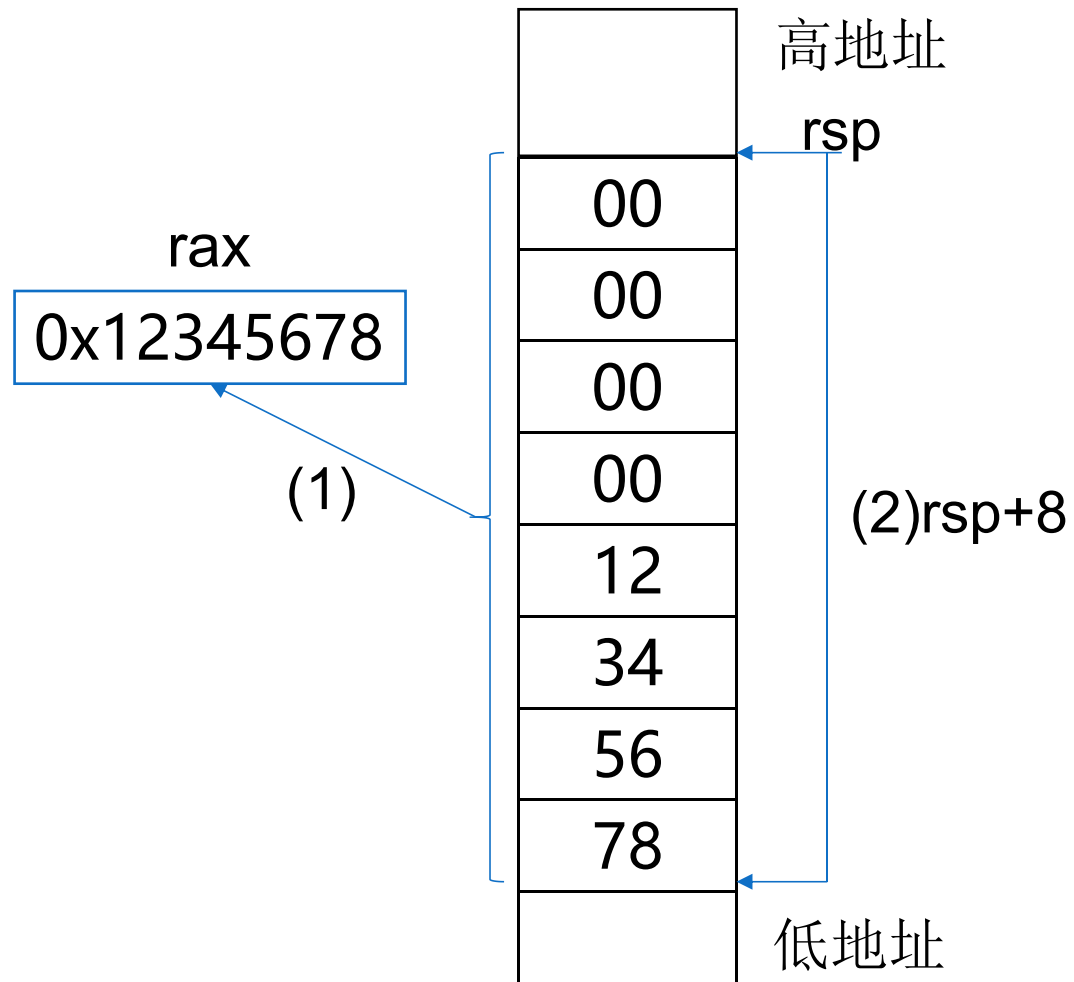
- 先将栈顶数据传送到目的操作数（通用寄存器、存储单元或段寄存器）
- 后ESP增加作为当前栈顶
- 以字或四字为单位操作
 - 出栈四字量数据时，RSP加8
 - 出栈字量数据时，RSP加2

pop rax

出栈操作



#进栈指令
pop rax
#等同于如下两条指令
mov rax, [rsp]
add rsp, 8



堆栈指令示例



#假设rsp= 0x00000052C01FFDE8

push rax	# rsp= 0x00000052C01FFDE0
pushf	# rsp=0x00000052C01FFDD8
mov rcx,[rsp]	# rcx=rflags=0x00000000000000202,
	# rsp=0x00000052C01FFDD8
mov rdx, [rsp+8]	# rdx=rax=0x0000000000000000A,
	# rsp=0x00000052C01FFDD8
popfq	# rsp=0x00000052C01FFDE0
pop rax	# rsp=0x00000052C01FFDE8